

# Neuro SAN

→ **Microsoft Agent Framework**

## Full Migration Guide

Architecture · Concept Mapping · Migration Process

MAF v1.0 · April 2026 · Confidential

**v1.0**

MAF stable  
April 2026

**1:1**

Concept mapping  
Neuro SAN → MAF

**0**

New concepts  
to learn

# Table of Contents

---

|    |                                           |                                           |
|----|-------------------------------------------|-------------------------------------------|
| 1. | <b>What is Neuro SAN?</b>                 | Understanding the baseline                |
| 2. | <b>What is MAF?</b>                       | The Microsoft Agent Framework explained   |
| 3. | <b>Key Terms Explained</b>                | HOCON, AAOSA, gRPC, Sly Data, and more    |
| 4. | <b>Concept Mapping: Neuro SAN → MAF</b>   | Every component mapped 1:1                |
| 5. | <b>Target Architecture</b>                | What the migrated system looks like       |
| 6. | <b>Migration Process</b>                  | The full end-to-end migration walkthrough |
| 7. | <b>Risks and Considerations</b>           | What to watch out for                     |
| 8. | <b>Recommendation &amp; Decision Asks</b> | What we need to move forward              |

---

**SECTION 1**

# What is Neuro SAN?

Neuro SAN (Neuro AI System of Agent Networks) is an open-source multi-agent orchestration framework built by Cognizant AI Lab. It allows developers and domain experts to build networks of AI agents that collaborate to solve complex tasks — entirely through declarative configuration files, with minimal coding required.

Think of it like building a company org chart for AI. Each agent is a specialist — one handles baggage queries, another handles refunds, another handles billing. A manager agent receives incoming requests and routes them to the right specialist automatically. The routing logic is built into the framework itself, not hardcoded by the developer.

## Core building blocks

### HOCON Config Files

The entire agent network is defined in a human-readable text file. Who each agent is, what it does, what tools it can call, how it connects to other agents — all in one place. No Python required to define the network.

### AAOSA Protocol

The built-in routing protocol. Agents decide among themselves who handles each part of a task. No hardcoded delegation paths — it emerges from the agents' reasoning.

### Coded Tools

Python classes that give agents real-world abilities: calling databases, APIs, or running calculations. Stored in a `coded_tools/` folder.

### Sly Data

A mechanism to pass sensitive information between agents without exposing it to the LLM. Passwords, API keys, and PII travel in a protected side-channel.

### NSFlow UI

An auto-included web dashboard on `localhost:4173` for interacting with and visualising agent networks.

### gRPC Server

The backend runtime that receives requests from the UI or API, routes them through the agent network, and returns results.

**Important context:** Neuro SAN is a Cognizant open-source project. It has no native Microsoft ecosystem integration, no Azure deployment path, no OpenTelemetry observability, and no roadmap alignment with MAF. This is the core reason for migration.

SECTION 2

# What is MAF?

Microsoft Agent Framework (MAF) is the unified, production-ready successor to both AutoGen and Semantic Kernel — two frameworks that Microsoft ran in parallel for years. Released in public preview in October 2025 and reaching v1.0 stable in April 2026, MAF merges the best of both into a single open-source SDK with long-term support commitment.

AutoGen brought dynamic multi-agent orchestration. Semantic Kernel brought enterprise-grade reliability — type safety, session state, filters, observability. MAF takes both, adds a new graph-based Workflow primitive, and deploys natively to Azure AI Foundry.

## The four pillars of MAF

|                                      |                                                                                                                                                                                                                           |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Open Standards & Interoperability    | Supports Model Context Protocol (MCP), Agent-to-Agent (A2A) messaging, and OpenAPI-first design. Agents built in MAF can talk to agents built in other frameworks and runtimes.                                           |
| Agent + Workflow Orchestration       | Two modes in one framework: Agent Orchestration for dynamic, LLM-driven reasoning (AutoGen DNA), and Workflow Orchestration for deterministic, graph-based business processes (SK DNA). You pick the right mode per task. |
| Enterprise-Grade Production Features | Built-in OpenTelemetry observability, Microsoft Entra ID authentication, PII detection, prompt injection shields, task adherence monitoring, and durable state for long-running agents.                                   |
| Azure-Native Deployment              | First-class integration with Azure AI Foundry for cloud deployment, Microsoft 365 Copilot, and the broader Microsoft agent ecosystem. KPMG, BMW, and Fujitsu are already running MAF in production.                       |

**Critical signal:** As of April 2026, AutoGen and Semantic Kernel have both entered **maintenance mode**. They receive bug fixes and security patches only — no new features. All Microsoft investment is now focused on MAF. This is the right time to migrate.

SECTION 3

# Key Terms Explained

This section explains each Neuro SAN-specific term in plain language, followed by the equivalent concept in MAF. This is the reference page for anyone new to either framework.

| HOCON                  | <i>Human-Optimized Config Object Notation</i>                                                                                                                                                                                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>In Neuro SAN:</b>   | A text-based configuration format (like a friendlier JSON). In Neuro SAN, you define your entire agent network in a .hocon file — each agent's name, role, instructions, which tools it has access to, and how it connects to other agents. No Python code is needed to define the network itself.                    |
| <b>MAF Equivalent:</b> | In MAF, agents are defined in Python code using the Agent class. You create an agent by instantiating it with a name, instructions (system prompt), and a list of tools. While this requires code instead of config, it unlocks type safety, unit testing, CI/CD pipelines, and version control of agent definitions. |

| AAOSA Protocol         | <i>Adaptive Agent-Oriented Software Architecture</i>                                                                                                                                                                                                                                                                                                                                               |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>In Neuro SAN:</b>   | The internal delegation protocol Neuro SAN uses for routing. When a user sends a request, the front-desk agent does not hardcode which specialist to call. Instead, each agent in the network evaluates whether it can contribute to the task, and delegation emerges from that collective reasoning. This makes routing flexible but also somewhat opaque — it can be hard to predict or debug.   |
| <b>MAF Equivalent:</b> | MAF replaces this with a Workflow Graph — a directed graph where you explicitly define which agent can hand off to which. This makes routing transparent and deterministic. You can still have dynamic LLM-driven selection within nodes, but the overall routing topology is visible and controllable. This is a significant improvement for debugging and governance in enterprise environments. |

| gRPC Server          | <i>Google Remote Procedure Call server</i>                                                                                                                                                                                                                                                                       |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>In Neuro SAN:</b> | The backend runtime layer in Neuro SAN. When the NSFlow UI or an external API sends a request, it hits the gRPC server first. The server manages the agent network lifecycle, routes requests to the right network, and streams back responses. It is essentially the engine room that keeps everything running. |

**MAF  
Equivalent:**

MAF provides a Hosted Agent runtime through Azure AI Foundry. Agents expose themselves via the A2A (Agent-to-Agent) protocol — a standardised communication layer that allows agents to be called from any client, any runtime, even across different frameworks. You no longer manage a gRPC server yourself; the runtime is handled by the cloud infrastructure.

**Sly Data**

*Neuro SAN's sensitive data side-channel*

**In Neuro SAN:**

A mechanism that lets sensitive data (PII, passwords, API keys, account numbers) travel between agents without ever being included in the text context that gets sent to the LLM. The LLM never sees this data — it is passed via a protected internal channel. This is critical for compliance in regulated industries. However, it relies on developer discipline — the framework does not actively enforce it.

**MAF  
Equivalent:**

MAF handles this through Session-based State combined with Content Filters. Sensitive data lives in the session state object, which tools can read but which never enters the LLM prompt. Additionally, MAF has built-in PII detection (via Azure AI Foundry), prompt injection shields, and task adherence monitoring — providing framework-enforced protection rather than relying on developer discipline.

**Coded Tools**

*Custom Python capabilities for agents*

**In Neuro SAN:**

Python classes stored in the coded\_tools/ folder that give agents real-world abilities beyond just reasoning. An agent with a lookup\_policy coded tool can actually query a database. An agent with a send\_email tool can dispatch notifications. Tools are the bridge between LLM reasoning and real systems.

**MAF  
Equivalent:**

In MAF, tools are simple async Python functions registered inline when creating an agent. There is no special folder structure or base class required — a Python function with a docstring is all you need. MAF also supports MCP (Model Context Protocol) tool servers, meaning you can plug in entire tool ecosystems from the MCP registry without writing custom code.

**NSFlow UI**

*Neuro SAN's built-in web dashboard*

**In Neuro SAN:**

A web interface that starts automatically when you run Neuro SAN (python -m run), available at localhost:4173. It provides a chat interface for interacting with agent networks, real-time visibility into agent "thinking" logs, and message flow visualisation. It is useful for local development and demos.

**MAF  
Equivalent:**

MAF has DevUI — currently in preview — which provides similar local development capabilities. For production, the Azure AI Foundry portal provides observability dashboards, conversation tracing, model performance metrics, and compliance reporting. The Foundry portal is significantly more capable than NSFlow for production workloads.

## SECTION 4

## Concept Mapping: Neuro SAN → MAF

Every concept in Neuro SAN has a direct equivalent in MAF. Nothing is lost in the migration — the same capabilities exist, just expressed differently. The table below maps each concept, identifies its MAF replacement, and specifies which layer of MAF it belongs to.

| Neuro SAN Concept                                                   | MAF Component                                                                          | MAF Layer                                                                       |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>HOCON agent file</b><br>Entire network in one .hocon text file   | <b>Agent class (agent.RunAsync)</b><br>Python class with name, instructions, tools     | <b>MAF Core — AutoGen DNA</b><br>The foundational agent abstraction             |
| <b>AAOSA protocol</b><br>Automatic emergent delegation routing      | <b>Workflow Graph (directed graph)</b><br>Explicit node-to-node routing topology       | <b>MAF Workflows — new primitive</b><br>Declarative, inspectable orchestration  |
| <b>Coded Tools (Python class)</b><br>Classes in coded_tools/ folder | <b>Tool function (inline async def)</b><br>Simple Python functions + MCP support       | <b>MAF Tools — SK Plugin DNA</b><br>No base class or folder structure needed    |
| <b>Sly Data</b><br>Protected side-channel for PII / secrets         | <b>Session state + content filters</b><br>Framework-enforced, not developer-discipline | <b>MAF State — SK enterprise DNA</b><br>PII detection + prompt shields built in |
| <b>NSFlow UI</b><br>Auto-included web UI at localhost:4173          | <b>DevUI + Azure Foundry portal</b><br>Local dev UI + production observability         | <b>MAF DevUI — in preview</b><br>Foundry provides production dashboards         |
| <b>gRPC server</b><br>Self-managed backend runtime                  | <b>Hosted agent + A2A runtime</b><br>Cloud-managed, cross-runtime comms                | <b>MAF Runtime — new primitive</b><br>No server management required             |
| <b>LLM config in HOCON</b><br>Model settings inside config file     | <b>Microsoft.Extensions.AI client</b><br>Unified provider-agnostic LLM client          | <b>MAF Foundation — unified layer</b><br>OpenAI, Azure, Anthropic, Ollama, etc. |



SECTION 5

# Target Architecture

This section describes what the Neuro SAN system looks like after full migration to MAF. The same multi-agent behaviour is preserved, but the underlying infrastructure is entirely replaced with MAF components. Each layer is explained in detail below.

## How a request flows through the migrated system

A user or API sends a request. The MAF Workflow Graph receives it and routes it to the Manager Agent. The Manager Agent decides — based on its instructions and LLM reasoning — which specialist agent to hand off to. The specialist agent uses its registered Tool functions to call real APIs, databases, or services. Sensitive data (PII, credentials) lives in the MAF Session State and is never passed into any LLM prompt. The entire conversation is traced in Azure AI Foundry via OpenTelemetry.

|                             |                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layer 1: User / API Request | The entry point. Can be a web UI, a REST API call, a Slack message, or a Microsoft 365 Copilot invocation. MAF's A2A protocol means any compliant client can call any MAF agent.                                                                                                                                                                                 |
| Layer 2: MAF Workflow Graph | Replaces AAOSA + HOCON orchestration. A directed graph that defines which agents exist, what they're responsible for, and which agents can hand off to which. Routing is explicit and visible — unlike AAOSA's emergent, opaque delegation. You can add conditional branches, parallel execution paths, and human-in-the-loop checkpoints as nodes in the graph. |
| Layer 3: Specialist Agents  | Individual agents — Manager, Baggage, Refund, Billing, etc. — each defined as a MAF Agent class with a system prompt (instructions) and a list of tools. These replace the HOCON agent definitions. Each agent is independently testable, version-controlled, and can be reused across multiple workflows.                                                       |
| Layer 4: MAF Tools          | Replaces coded_tools/. Each tool is an async Python function that the agent can invoke when needed. Tools interact with databases, APIs, or external services. MAF also supports MCP tool servers, allowing you to connect to standardised tool registries without writing integration code.                                                                     |

|                                   |                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Layer 5: MAF Session State</b> | Replaces Sly Data. The session state object stores sensitive data across the conversation lifecycle. It never enters any LLM prompt. Additionally, MAF's content filters provide PII detection, prompt injection shields, and task adherence monitoring — all enforced by the framework, not by developer discipline.     |
| <b>Layer 6: Azure AI Foundry</b>  | The deployment and observability layer. Provides OpenTelemetry-based distributed tracing, real-time conversation monitoring, model performance dashboards, Entra ID authentication, compliance reporting, and one-click cloud deployment. Replaces the self-managed gRPC server and NSFlow UI in production environments. |

## SECTION 6

# Migration Process

This section describes the full end-to-end process for migrating Neuro SAN to MAF. It is structured as four sequential phases. Each phase has a clear starting point, a set of activities, and a concrete outcome that signals it is complete before the next begins.

## Phase 1 — Environment Setup and Proof of Concept

**Goal:** Install MAF and prove that one complete Neuro SAN agent network can be reproduced end-to-end in MAF before touching anything in production.

### What happens in this phase

The team starts by installing the MAF Python package and setting up a local development environment. The recommended approach is to pick the simplest existing Neuro SAN agent network — ideally the Airline Policy example — and rebuild it from scratch in MAF. This forces every concept to be translated at least once, and surfaces any unexpected gaps before they become blockers.

The HOCON file for the chosen network is the specification document. Each agent block in HOCON becomes one MAF Agent class. The agent's instructions field maps directly to the `system_message` or `instructions` parameter. The tools list maps to the `tools=` parameter. The network topology (which agents can delegate to which) becomes the edges of the MAF Workflow Graph.

The PoC is validated by running the same set of test queries against both the original Neuro SAN network and the MAF version, and comparing outputs. The PoC is complete when the MAF version produces equivalent responses.

### Key installation steps

```
# Install MAF (Python)
pip install agent-framework
pip install agent-framework-azure-ai # for Azure integration

# Or install components individually
pip install agent-framework-core
pip install agent-framework-workflows

# Set environment variables
export OPENAI_API_KEY="your-key"
export AZURE_OPENAI_ENDPOINT="https://your-resource.openai.azure.com/"
```

### What a converted agent looks like

Before (Neuro SAN HOCON):

```
{
  name: "refund_agent"
  instructions: "You are a refund specialist. Use your tools to check eligibility."
  tools: ["check_refund_eligibility", "process_refund"]
}
```

After (MAF Python):

```
from agent_framework import Agent
refund_agent = Agent(
  name="refund_agent",
  instructions="You are a refund specialist. Use your tools to check eligibility.",
  tools=[check_refund_eligibility, process_refund]
)
```

**Phase 1 is complete when:** The PoC agent network runs in MAF, produces equivalent outputs to the Neuro SAN version, and the team has validated the full HOCON-to-Agent mapping.

## Phase 2 — Tools and State Migration

**Goal:** Migrate all coded tools from Neuro SAN's Python classes to MAF tool functions, and replace Sly Data with MAF's session state and content filters.

### What happens in this phase

Each Python class in the `coded_tools/` folder is converted to a simple async Python function. The logic inside the class method stays exactly the same — only the wrapping changes. The function's docstring becomes the tool's description that the LLM reads when deciding whether to call it. Type annotations on the function parameters become the tool's input schema.

Sly Data is replaced by moving all sensitive data (PII, API keys, session tokens, account numbers) into the MAF session state object. Tools read from this object directly — they never pass sensitive data through the agent's text conversation. Additionally, MAF's content filter pipeline is configured to provide PII detection, prompt injection shields, and task adherence monitoring at the framework level.

### Tool conversion example

Before (Neuro SAN coded tool):

```
class CheckRefundEligibility:
  def invoke(self, booking_id: str) -> dict:
    """Check if a booking is eligible for refund."""
    result = db.query(f"SELECT * FROM bookings WHERE id={booking_id}")
    return {"eligible": result.days_since_booking <= 3}
```

After (MAF tool function):

```
async def check_refund_eligibility(booking_id: str) -> dict:
    """Check if a booking is eligible for refund."""
    result = db.query(f"SELECT * FROM bookings WHERE id={booking_id}")
    return {"eligible": result.days_since_booking <= 3}
```

**Phase 2 is complete when:** All coded\_tools/ classes are migrated to MAF functions, all Sly Data patterns are replaced with session state, and content filters are configured and validated against test PII inputs.

## Phase 3 — Orchestration Migration

**Goal:** Replace AAOSA's emergent delegation with a MAF Workflow Graph. Make the entire agent routing topology explicit, testable, and governable.

### What happens in this phase

This is the most architecturally significant phase. In Neuro SAN, the HOCON tools list implicitly defines which agents can communicate with which — and AAOSA decides at runtime who handles what. In MAF, this topology is made explicit as a Workflow Graph.

The team reads each HOCON agent definition and extracts its tools list. Any tool that is another agent (rather than a coded tool) becomes an edge in the Workflow Graph. The manager agent that delegates to specialist agents becomes the root node, and the specialist agents become child nodes. The graph defines which agent can transfer control to which, under what conditions.

Within each node, the routing can still be LLM-driven — an agent can use its reasoning to decide whether to handle a task or hand it off. The difference is that the set of possible handoff destinations is explicitly declared in the graph, not implicit and emergent. This gives the architecture team full visibility into agent communication topology.

This phase also includes adding Human-in-the-Loop (HITL) checkpoints where required — for example, before an agent executes a high-value refund, a human approval node can be inserted into the graph. This is a new capability that Neuro SAN does not natively support.

**Phase 3 is complete when:** All agent networks are running on MAF Workflow Graphs, the routing topology is documented and reviewed by the architecture team, and any human-in-the-loop checkpoints required for compliance have been implemented and tested.

## Phase 4 — Azure Deployment and Decommission

**Goal:** Deploy all migrated agent networks to Azure AI Foundry, enable enterprise security and observability, and shut down the Neuro SAN gRPC server.

### What happens in this phase

With all agent networks validated locally on MAF, the team deploys to Azure AI Foundry. This involves connecting each agent to the Foundry Agent Service, which provides managed hosting, auto-scaling, and a production API endpoint.

Microsoft Entra ID authentication is configured so that all API calls to the agents are authenticated via Azure AD — replacing any custom auth logic in the Neuro SAN gRPC layer. OpenTelemetry traces are activated, flowing agent conversation data into the Foundry observability dashboard. This gives the operations team real-time visibility into agent performance, error rates, and response latencies.

Once the Foundry-hosted agents are validated in production traffic, the Neuro SAN gRPC server is decommissioned. All clients (web UIs, APIs, other services) are updated to point to the new MAF endpoints. The NSFlow UI is replaced by the Azure Foundry portal for monitoring, and the MAF DevUI for local development.

**Phase 4 is complete when:** All agents are live on Azure AI Foundry, Entra ID auth and OpenTelemetry observability are active, production traffic is flowing through MAF endpoints, and the Neuro SAN gRPC server has been shut down.

SECTION 7

# Risks and Considerations

Every migration carries risk. The following are the key risks identified for this migration, along with their likelihood, impact, and mitigation strategy.

| Risk                             | Likelihood | Impact | Mitigation                                                                                                                                                                                                                     |
|----------------------------------|------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AAOSA behaviour differences      | Medium     | Medium | AAOSA's emergent delegation may produce different routing decisions than a MAF Workflow Graph for the same input. The PoC phase (Phase 1) is specifically designed to catch this early through side-by-side output comparison. |
| Sly Data pattern coverage        | Low        | High   | If any Sly Data usage has been missed during audit, sensitive data could end up in LLM prompts. A full audit of all data flows must be completed before Phase 2 is signed off.                                                 |
| MAF DevUI still in preview       | Low        | Low    | The local development UI for MAF is in preview and may change. This does not affect production — Azure Foundry portal covers production needs. Risk is limited to developer experience.                                        |
| Azure subscription costs         | Medium     | Medium | Azure AI Foundry Agent Service has associated costs. A cost estimate should be produced before Phase 4 begins, based on expected request volumes.                                                                              |
| Team reskilling (HOCON → Python) | Medium     | Low    | Domain experts who currently edit HOCON files directly will need to either learn Python or use AutoGen Studio for prototyping. A short internal training session covers this adequately.                                       |

## SECTION 8

## Recommendation & Decision Asks

**Recommendation: Fully migrate Neuro SAN to MAF v1.0** MAF v1.0 is production-ready with a long-term Microsoft support commitment. Every Neuro SAN concept maps 1:1 to a MAF equivalent — there is nothing to re-learn, only a change in how existing concepts are expressed. AutoGen and Semantic Kernel are now in maintenance mode, and Neuro SAN has no Microsoft ecosystem roadmap. The migration window is now.

### Decision asks

1

**Approve the migration process**

Sign off on the four-phase migration approach described in Section 6 as the delivery plan.

2

**Allocate a PoC team**

2 engineers for Phase 1 (2 weeks). This validates the migration approach before full commitment.

3

**Azure AI Foundry subscription access**

Required for Phase 4 deployment. Cost estimation should be completed before Phase 4 begins.

4

**Sign-off to decommission Neuro SAN gRPC server**

Formal approval to shut down the Neuro SAN backend once Phase 4 validation is complete.